Jacob Yankee
COSC 341
Zhang

I. Reference
- I have used this github code for reference when writing number 6 https://github.com/thennbreak/SML-Function-for-Duplicate-Deletion/blob/master/dupDelete.sml

II. Design
- For product(), I made sure it was possible for either n or m to be negative
- For delnthc(), I wrote it with the intent to use it as implode(delnthc(explode("string", n)));
- For dispnthc(), I wrote it with the intent to use it as dispnthc(explode("string"),n);
- For pairStar(), I wrote it with the intent to use it as implode(pairStar(explode("string")));
- For int2str() I could not find a way to use only the int as input so it takes in both the int and an int list that results from the a helper function int2list. Written with the intent to use it as implode(int2str(n, int2list(n)));
- For str2int(), I wrote it with the intent to use it as str2int(explode("string")); where the string is some number
- For toupp(), I wrote it with the intent to use it as implode(toupp(explode("string"))); Furthermore I made sure that it could handle strings of any input

III. Difficulty
- Most of these weren't too difficult, most of the challenge comes from the archaic nature of SML and could've been done much easier using any other language
- Sometimes I would run into timeout exceptions while writing certain functions
- I could not figure out how to convert one helper function from if-then-else to pattern

IV. Comments
- The work is more obnoxious than difficult due to the syntax of things
- The professor's sample report helps to nail that idea in with how negative some of his comments are. Are they supposed to be satirical?

Appendix One – Source Code

If-Then-Else Form:

```
(* Function 1 *)

(* Function name: product *)
(* Description: Computes the product of two integers *)
fun product(m, n) =
if m = 0
then 0
else
   if m < 0
```

```
    then ~n + product(m+1, n)
    else n + product(m-1, n);

(* Function 2 *)

(* Function name: delnthc *)
(* Description: Deletes the n-th character in a string *)
fun delnthc(L, n) =
if null L
then nil
else
    if n = 1
    then delnthc(tl L, n-1)
    else hd L :: delnthc(tl L, n-1);

(* Function 3 *)

(* Function name: dispnthc *)
(* Description: Displays the n-th character in a string *)
fun dispnthc(L, n) =
if n = 1
then hd L
else dispnthc(tl L, n-1);

(* Function 4 *)

(* Function name: pairStar *)
(* Description: Forms a new string where adjacent identical characters are separated by a * *)
fun pairStar(L) =
if null L
then nil
else
    if null (tl L)
    then hd L :: nil
    else
        if hd L = hd(tl L)
        then hd L :: #"*" :: pairStar(tl L)
        else hd L :: pairStar(tl L);

(* Function 5 *)

(* Function name: remv *)
(* Description: Removes elements from list L that match value k *)
fun remv(k, L) =
```

```
if null L
then nil
else
    if k = hd(L)
    then remv(k, tl L)
    else hd(L) :: remv(k, tl L);


(* Function 6 *)

(* Function name: compare *)
(* Function description: Helper function to compare if values in a list are equal *)
fun compare(n, L) =
if null L
then nil
else
    if n = hd L
    then compare(n, tl L)
    else hd L :: compare(n, tl L);

(* Function name: remvdub *)
(* Description: Removes duplicate values from a list *)
fun remvdub(L) =
if null L
then nil
else hd L :: remvdub(compare(hd L, tl L));

(* Function 7 *)

(* Function name: inc1 *)
(* Description: Increases the value of a second element in a tuple by one if the first element is
the same as the given value *)
fun inc1(n, L:(int * int) list) =
if null L
then nil
else
    if #1(hd L) = n
    then (#1(hd L), #2(hd L) + 1) :: inc1(n, tl L)
    else hd L :: inc1(n, tl L);

(* Function 8 *)

(* Function name: min2 *)
(* Description: Finds the second smallest number in an integer list *)
```

```
fun min2(L) =
if hd L < hd(tl L)
then
   if hd(tl L) < hd(tl (tl L))
   then hd(tl L)
   else min2((hd L) :: tl(tl L))
else
   if hd L < hd(tl(tl L))
   then hd L
   else min2(tl(tl L));

(* Function 9 *)

(* Function name: app *)
(* Description: A function that substitutes for @ *)
fun app(x, y)=
if null x
then y
else hd x :: app(tl x, y);

(* Function name: int2list *)
(* Description: Takes an int and converts its digits to an int list *)
fun int2list(n) =
if n < 0
then int2list(~n)
else
if n < 10
then n :: nil
else app(int2list(n div 10), (n mod 10) :: nil);

(* Function name: int2str *)
(* Description: Converts an int list to a char list. Takes in both int and int list to determine
whether or not a negative is needed. Returning as a string requires the use of implode in
function call *)
   fun int2str(n, L) =
if null L
then nil
else
   if n < 0
   then #"~" :: int2str(~n, L)
   else chr(hd L + 48) :: int2str(n, tl L);

(* Function 10 *)
```

```
(* Function name: char2num *)
(* Description: Converts a char 0-9 to int 0-9 *)
fun char2num(n) =
if n < 10
then 0
else (n - 48);

(* Function name: len *)
(* Description: Finds the length of a list *)
fun len(L) =
if null L
then 0
else len(tl L) + 1;

(* Function name: exp *)
(* Description: Calculates an exponent *)
fun exp(n, m) =
if m = 0
then 1
else n * exp(n, m-1);

(* Function name: str2int *)
(* Description: Converts a char list to an int. Using a string as input requires the use of explode
in function call *)
fun str2int(L) =
if null L
then 0
else
    if hd L = #"~"
    then ~1 * str2int(tl L)
    else (char2num(ord(hd L)) * exp(10, len(tl L)))+ str2int(tl L);

(* Function 11 *)

(* Function name: toupp *)
(* Description: Intakes a char list and converts lowercase letters to uppercase. using a string
requires the use of both implode and explode in function call *)
  fun toupp(L) =
if null L
then nil
else
    if ord(hd L) > 96 andalso ord(hd L) < 123
    then chr(ord (hd L) - 32) :: toupp(tl L)
    else chr(ord (hd L)) :: toupp(tl L);
```

```
(* Function 12 *)

(* Function name: power *)
(* Description: Calculates an exponent *)
fun power(x, y) =
if y = 0
then 1
else x * power(x, y - 1);

(* Function name: multinHelp *)
(* Description: A helper class that creates a list *)
fun multinHelp(a, b, c, d) =
if d = c
then a * power(b, d) :: nil
else a * power(b, d) :: multinHelp(a, b, c, d + 1);

(* Function name: multin *)
(* Description: Takes list of three [a,b,c] and multiplies a by b c times, returns a list  of [a*b^0,
a*b^1, ..., a*b^c] *)
fun multin(L) =
if null L
then nil
else
	let
		val a = hd L
		val b = hd(tl L)
		val c = hd(tl(tl L))
		val output = multinHelp(a,b,c,0)
	in
		output
	end;

Pattern Form:
(* Jacob Yankee *)
(* COSC 341 *)
(* Zhang *)
(* Homework 1 *)

(* Function 1 *)

(* Function name: product *)
(* Description: Computes the product of two integers *)
fun product (0, _) = 0
```

```
   | product (m, n) =
    if m < 0 then
      ~n + product (m + 1, n)
    else
      n + product (m - 1, n);

(* Function 2 *)

(* Function name: delnthc *)
(* Description: Deletes the n-th character in a string *)
fun delnthc (nil, _) = nil
  | delnthc (x::xs, 1) = delnthc (xs, 0)
  | delnthc (x::xs, n) = x :: delnthc (xs, n - 1);

(* Function 3 *)

(* Function name: dispnthc *)
(* Description: Displays the n-th character in a string *)
fun dispnthc (x :: _, 1) = x
  | dispnthc (_ :: xs, n) = dispnthc (xs, n - 1);

(* Function 4 *)

(* Function name: pairStar *)
(* Description: Forms a new string where adjacent identical characters are separated by a * *)
fun pairStar(nil) = nil
  | pairStar [x] = [x]
  | pairStar (x :: y :: xs) =
      if x = y
      then x :: #"*" :: pairStar (y :: xs)
      else x :: pairStar (y :: xs);

(* Function 5 *)

(* Function name: remv *)
(* Description: Removes elements from list L that match value k *)
fun remv(_, nil) = nil
  | remv(k, x::xs) =
    if k = x then remv(k, xs)
    else x :: remv(k, xs);


(* Function 6 *)
```

```
(* Function name: compare *)
(* Function description: Helper function to compare if values in a list are equal *)
    fun compare(_, nil) = nil
        | compare(n, x::xs) =
        if n = x
        then compare(n, xs)
        else x :: compare(n, xs);

(* Function name: remvdub *)
(* Description: Removes duplicate values from a list *)
fun remvdub(nil) = nil
  | remvdub(x::xs) = x :: remvdub(compare(x, xs));

(* Function 7 *)

(* Function name: inc1 *)
(* Description: Increases the value of a second element in a tuple by one if the first element is
the same as the given value *)
fun inc1(n, nil) = nil
  | inc1(n, (x, y) :: tl) =
    if x = n
    then (x, y + 1) :: inc1(n, tl)
    else (x, y) :: inc1(n, tl);

(* Function 8 *)

(* Function name: min2 *)
(* Description: Finds the second smallest number in an integer list *)
fun min2(nil) = 0
    | min2(x::y::z::zs) = if x < y andalso x < z
    then
        if y < z
        then y
        else min2(x::z::zs)
    else if y < z then min2(x::y::zs)
    else min2(x::z::zs);

(* Function 9 *)

(* Function name: app *)
(* Description: A function that substitutes for @ *)
fun app(nil, y) = y
  | app(x, y) = hd x :: app(tl x, y);
```

```
(* Function name: int2list *)
(* Description: Takes an int and converts its digits to an int list *)
fun int2list(n) =
if n < 0
then int2list(~n)
else
    if n < 10
    then n :: nil
    else app(int2list(n div 10), (n mod 10) :: nil);

(* Function name: int2str *)
(* Description: Converts an int list to a char list. Takes in both int and int list to determine
whether or not a negative is needed. Returning as a string requires the use of implode in
function call *)
fun int2str(n, L) =
if null L
then nil
else
    if n < 0
    then #"~" :: int2str(~n, L)
    else chr(hd L + 48) :: int2str(n, tl L);

(* Function 10 *)

(* Function name: char2num *)
(* Description: Converts a char 0-9 to int 0-9 *)
fun char2num n =
  if n < 10 then 0 else n - 48;

(* Function name: len *)
(* Description: Finds the length of a list *)
fun len(nil) = 0
  | len (x::xs) = len xs + 1;

(* Function name: exp *)
(* Description: Calculates an exponent *)
fun exp (n, 0) = 1
  | exp (n, m) = n * exp (n, m - 1);

(* Function name: str2int *)
(* Description: Converts a char list to an int. Using a string as input requires the use of explode
in function call *)
    fun str2int(nil) = 0
  | str2int (#"~"::xs) = ~1 * str2int xs
```

```
  | str2int (x::xs) = (char2num (ord x) * exp (10, len xs)) + str2int xs;

(* Function 11 *)

(* Function name: toupp *)
(* Description: Intakes a char list and converts lowercase letters to uppercase. using a string
requires the use of both implode and explode in function call *)
fun toupp(nil) = nil
  | toupp(x::xs) =
    if ord(x) > 96 andalso ord(x) < 123
    then chr(ord x - 32) :: toupp(xs)
    else chr(ord x) :: toupp(xs);

(* Function 12 *)

(* Function name: power *)
(* Description: Calculates an exponent *)
fun power(_, 0) = 1
  | power(x, y) = x * power(x, y - 1);

(* Function name: multinHelp *)
(* Description: A helper class that creates a list *)
fun multinHelp(_, _, 0, _) = nil
  | multinHelp(a, b, c, d) =
      if d = c
      then [a * power(b, d)]
      else a * power(b, d) :: multinHelp(a, b, c, d + 1);

(* Function name: multin *)
(* Description: Takes list of three [a,b,c] and multiplies a by b c times, returns a list  of [a*b^0,
a*b^1, ..., a*b^c] *)

fun multin(nil) = nil
  | multin(x::y::z::zs) =
      let
        val a = x
        val b = y
        val c = z
        val output = multinHelp(a, b, c, 0)
      in
        output
      end;
```

Appendix Two – Sample Runs

Function 1:
Inputs:

```
product(2, 3);
product(~2, 3);
product(2, ~3);
product(~2, ~3);
```

Outputs:
val it = 6 : int
val it = ~6 : int
val it = ~6 : int
val it = 6 : int

Function 2:
Inputs:

```
implode(delnthc(explode("abcdef"), 4));
implode(delnthc(explode("hihihi"), 2));
```

Outputs:
val it = "abcef": string
val it = "hhihi": string

Function 3:
Inputs:

```
dispnthc(explode("abcdef"), 4);
dispnthc(explode("kevin12"), 6);
```

Outputs:
val it = #”d”: char
val it = #”1”: char

Function 4:
Inputs:

```
implode(pairStar(explode("xxyy")));
implode(pairStar(explode("aaaa")));
```

Outputs:
val it = “x*xy*y”: string
val it = “a*a*a*a”: string

Function 5:

Inputs:
remv("a", ["a", "b", "a", "c"]);
remv("4", ["1", "f", "4", "4"]);

Outputs:
val it = [“b”, “c”]: string list
val it = [“1”, “f”]: string list

Function 6:
Inputs:
remvdub(["a", "b", "a", "c", "b", "a"]);
remvdub(["a", "1", "1", "1", "b"]);

Outputs:
val it = [“a”, “b”, “c”]: string list
val it = [“a’”, “1”, “b”]: string list

Function 7:
Inputs:
inc1(2, [(1,1), (2,4), (4,5), (2,1)]);
inc1(4, [(4,4), (3,4), (4,1), (1,1)]);

Outputs:
val it = [(1,1), (2,5), (4,5), (2,2)]: (int * int) list
val it = [(4,5), (3,4), (4,2), (1,1)]: (int * int) list

Function 8:
Inputs:
min2([1,3,2,5,4]);
min2([3,12,6,4,9]);

Outputs:
val it = 2: int
val it = 4: int

Function 9:
Inputs:
implode(int2str(1234, int2list(1234)));
implode(int2str(~1234, int2list(~1234)));

Outputs:
val it = “1234”: string
val it = “~1234”: string

Function 10:
Inputs:
str2int(explode("1234"));
str2int(explode("~1234"));

Outputs:
val it = 1234: int
val it = ~1234: int

Function 11:
Inputs:
implode(toupp(explode("programming")));
implode(toupp(explode("Abcd123!")));

Outputs:
val it = “PROGRAMMING”: string
val it = “ABCD123!”: string

Function 12:
Inputs:
multin([2,3,5]);
multin([3,4,5]);

Outputs:
val it = [2,6,18,54,162,486]: int list
val it = [3,12,48,192,768,3072]:int list